

(Q1)

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def preprocess_and_binarize(image_path):
```

```
    # Let's Load the grayscale image
```

```
    img = cv2.imread('tools.pgm', cv2.IMREAD_GRAYSCALE)
```

```
    if img is None:
```

```
        print("Error: Image not found.")
```

```
        return None
```

```
    #Let's Apply Gaussian blur to reduce noise
```

```
    blurred = cv2.GaussianBlur(img, (5, 5), 0)
```

```
    # Let's Use Otsu's thresholding for automatic binary conversion
```

```
    _, binary_img = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY +  
cv2.THRESH_OTSU)
```

```
    return binary_img
```

```
def display_image(image, title="Binary Image"):
```

```
    plt.figure(figsize=(6,6))
```

```
    plt.imshow(image, cmap='gray')
```

```
    plt.title(title)
```

```
    plt.axis('off')
```

```
plt.show()
```

```
# Let's Define the image path (Ensure tools.pgm is in the working directory)
```

```
image_path = "tools.pgm"
```

```
# Let's Convert image to binary
```

```
binary_image = preprocess_and_binarize(image_path)
```

```
# Let's Display the binary image
```

```
if binary_image is not None:
```

```
    display_image(binary_image) (Q2)
```

```
import cv2
```

```
import numpy as np
```

```
from collections import deque
```

```
def floodfill(rgb_image, binary_image, start_i, start_j, color, visited):
```

```
    """Custom floodfill using BFS to color a connected region."""
```

```
    queue = deque()
```

```
    height, width = binary_image.shape
```

```
    # Let's Check if the start pixel is foreground and unvisited
```

```
    if binary_image[start_i, start_j] != 255 or visited[start_i, start_j]:
```

```
        return
```

```
    queue.append((start_i, start_j))
```

```
    visited[start_i, start_j] = True
```

```
    rgb_image[start_i, start_j] = color
```

```

# Let's Define 4-connectivity neighbors (up, down, left, right)
neighbors = [(-1, 0), (1, 0), (0, -1), (0, 1)]

while queue:
    i, j = queue.popleft()
    for di, dj in neighbors:
        ni, nj = i + di, j + dj
        # Let's Check boundaries
        if 0 <= ni < height and 0 <= nj < width:
            if binary_image[ni, nj] == 255 and not visited[ni, nj]:
                visited[ni, nj] = True
                rgb_image[ni, nj] = color
                queue.append((ni, nj))

# Let's Read the binary image from Part I
binary_image = cv2.imread('binary_tools.png', cv2.IMREAD_GRAYSCALE)
if binary_image is None:
    print("Error: Could not read the binary image.")
    exit()

height, width = binary_image.shape
visited = np.zeros((height, width), dtype=bool)
rgb_image = np.zeros((height, width, 3), dtype=np.uint8)

# Let's Predefined colors for three tools (red, green, blue)
colors = [

```

```

(255, 0, 0), # Red
(0, 255, 0), # Green
(0, 0, 255) # Blue
]
color_index = 0

# Let's Iterate through every pixel
for i in range(height):
    for j in range(width):
        if binary_image[i, j] == 255 and not visited[i, j]:
            if color_index >= len(colors):
                print("Warning: More objects detected than predefined colors!")
                continue
            floodfill(rgb_image, binary_image, i, j, colors[color_index], visited)
            color_index += 1

# Let's Save the result
cv2.imwrite('colored_tools.png', rgb_image)

# Let's Display the result (optional)
from matplotlib import pyplot as plt
plt.imshow(rgb_image)
plt.axis('off')
plt.show()

```

Results & Justification

Were the three tools colored individually?

Yes, provided the binary image from Part I successfully isolated the three tools into separate connected components. The floodfill algorithm works as follows:

1. Connected Component Detection: The algorithm traverses all pixels connected to a seed point, marking them as part of the same object.
2. Color Assignment: Each detected object is assigned a unique color from the predefined list.

Key Assumptions:

- The binary image has exactly three disconnected regions (one for each tool).
- The tools do not touch each other in the binary image (ensured by proper preprocessing in Part I).

Why It Works:

- Floodfill correctly identifies each tool as a separate region if they are not connected.
- Predefined colors ensure distinct coloring for up to three objects.